

Architecture, Performance, Rebuild Roadmap, and Unknowns

Spec-set role: forensic reconstruction document generated from the three recovered HOTAS Control Panel chat notes. This document is meant to stand on its own while cross-referencing the rest of the set.

Evidence taxonomy used throughout:

- ? Confirmed Facts: directly stated in the recovery notes or visible as exact recovered UI text.
- ? Screenshot-Derived Details: observations recovered from screenshot descriptions in the source notes.
- ? Inferences: explicitly inferred from the notes or reconstructed from adjacent evidence.
- ? Desired Improvements: user preferences, future direction, fixes, or rebuild requirements.
- ? Unknowns: details the notes say are missing, uncertain, or need later verification.

Confirmed Facts

- ? V2 is a safe parallel app, not a destructive rewrite.
- ? Chosen V2 frontend stack in the broad rebuild/productization source: PySide6 / Qt Widgets.
- ? Chosen graph stack in that source: pyqtgraph.
- ? Legacy app files mentioned:
 - ? hotas_control_panel_app.py
 - ? app_performance.py
 - ? hotas_ui_theme.py
 - ? Open-HOTAS-Control-Panel.cmd
 - ? Edit-HOTAS-Curves.cmd
 - ? HOTASControlPanel.exe
- ? V2/source/workspace path examples:
 - ? hotas_bridge_config_v2.json
 - ? v2_app/
 - ? shared_core/
 - ? legacy_app/
 - ? packaging/
 - ? assets/app_icon.ico
- ? User-data target locations include %AppData%\HOTASControlPanelV2 or %LocalAppData%\HOTASControlPanelV2.
- ? Installer target locations include %ProgramFiles%\HOTAS Control Panel V2 or %LocalAppData%\Programs\HOTAS Control Panel V2.

Screenshot-Derived Details

- ? Perf / Diagnostics page includes active page, runtime state, selected axis, source file, hidden page skips, runtime path checks, page build/switch times, heartbeat timing, graph draw timings, startup total/config/widget timing.
- ? Example visible diagnostics include v2.active_page: perf, v2.runtime: live, v2.selected_axis: Yaw, v2.source: V2 draft: hotas_bridge_config_v2.json, and v2.hidden_page_skips: 41689.
- ? Help / Docs page includes searchable topics and built-in product documentation.
- ? Legacy Performance popup included Perf On, Collect live timings, Clear, visible tab, selected axis, runtime timestamp, pending refresh counts, and scheduler tick data.

Inferences

- ? Runtime bridge likely reads physical HOTAS input, applies mapping/tuning/transforms, then emits final output via vJoy.
- ? Shared core should own config parsing, tuning math, rule engine, runtime bridge adapters, common models/DTOs, profiles, and workspace state.
- ? UI shell should use retained/lazy page construction, reused page instances, scoped graph updates, hidden-page throttling/skips, and central heartbeat/scheduler.
- ? Flight Recorder and Live Overlay should share telemetry/overlay core while using different adapters.

Desired Improvements

- ? Preserve legacy app until V2 compatibility is proven.
- ? Avoid whole-page redraw and layout churn.
- ? Keep graph updates local and cached.
- ? Hidden/non-live areas must skip expensive work.
- ? Use performance diagnostics to verify claims.
- ? Package as real Windows app with installer, Start Menu shortcut, optional Desktop shortcut, uninstaller, real .ico, and user data stored separately from binaries.
- ? Produce actual .ico files when asked, not only generated image previews.

Unknowns

- ? Exact original source code.
- ? Exact runtime bridge implementation.
- ? Exact vJoy library/API calls.
- ? Exact physical HOTAS polling method.
- ? Exact full schema for hotas_bridge_config_v2.json.
- ? Exact final dependency versions.
- ? Exact packaging scripts.
- ? Exact Helm diagnostic algorithm.
- ? Exact profile and rule schemas.

Rebuild Roadmap Summary

Confirmed Facts

1. Preserve legacy app.
2. Extract/define shared core interfaces.
3. Build V2 Qt shell.
4. Build design system/QSS.
5. Implement mapping/workspace/profile state.
6. Implement tuning pages and graphs.
7. Implement rules/stack/live monitor.
8. Implement Helm overlay.
9. Implement Help/Docs/Perf.
10. Add polish/copy/animations.
11. Package with installer and icon.
12. Test end-to-end.

Implementation Prompt Summary

Desired Improvements

Source	Prompt Scope	Where Full/Relevant Prompt Is Preserved
----	----	----
Rebuild/productization chat Full V2 rebuild, pages, safe workspace, graphs, Helm, packaging, tests This document, source-preserved block `## 14.. Codex / AT Rebuild Prompt`		
Flight recorder/live overlay chat Flight Recorder, Live Monitor, detached Live Overlay, Configure dialog, hindsight capture Document 7.. source-preserved block `## 14.. Codex / AT Rebuild Prompt`		
Helm/recovery chat Core tuning tabs, Conditional Rules, Effective Response Stack, Helm, performance, visual modernization Relevant excerpts in Documents 3-6; roadmap details preserved below		

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 2.2
Phase 1 performance/refactor work

2.2 Phase 1 performance/refactor work

What changed: a major under-the-hood refactor before the later UI work.

Preserved details from Codex report:

- ? UI/dirty/runtime updates moved through a coordinated heartbeat instead of competing refresh timers.
- ? Runtime polling became adaptive.
- ? Hidden and non-live areas were no longer polled at the same cost as live stack/rules views.
- ? Dirty-state tracking was reduced from whole-app snapshot churn to targeted per-tab refresh work.
- ? Preview math caching was introduced for linear references, static curves, dynamic previews, and feel/guidance output.
- ? Packaged app rebuilt to HOTASControlPanel.exe.
- ? Both launchers updated to prefer the packaged build:
 - ? Open-HOTAS-Control-Panel.cmd
 - ? Edit-HOTAS-Curves.cmd

Purpose: reduce app refresh cost, reduce UI churn, and improve responsiveness before UI polish.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 2.3
Phase 2 / advanced legacy visual pass

2.3 Phase 2 / advanced legacy visual pass

What it attempted: make the legacy UI bolder, more modern, and more visually polished.

What went wrong:

- ? UI looked only slightly more modern.
- ? Rounded corners were pixelated/jagged.
- ? Opening tabs became slower.
- ? Scrollbars/page scrolling broke on many pages.
- ? Content became unreachable.
- ? Sections continued overlapping.
- ? Bottom action bar consumed too much space and visually covered content.
- ? Too much decorative chrome stole room from actual controls.

User reaction: frustration; the pass looked like a partial facelift but introduced major usability regressions.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 2.4
Legacy repair/stabilization pass

2.4 Legacy repair/stabilization pass

Codex later repaired many legacy issues.

Root causes found:

- ? Global mousewheel handling used `bind_all()` once per `ScrollableFrame`, so every wheel event fanned out through multiple scroll containers.
- ? Modes, Base Tuning, Filtering, and Combat Profile lacked reliable page-level vertical scrolling.
- ? Footer/action bar was oversized and reduced usable viewport height.
- ? Too many major surfaces used a custom canvas-backed rounded renderer, adding redraw cost and jagged Windows scaling.

- ? Extra wrapper chrome consumed usable space.
- ? Hidden refresh work was already reduced, but there was not enough instrumentation for tab reuse/skips/scrollregion churn.

Fixes implemented:

- ? Simplified main shell layout.
- ? Workspace owned expanding middle row.
- ? Footer placed in its own reserved row.
- ? Removed "Tabs stay focused..." strip.
- ? Reduced header/workspace padding.
- ? Slimmed bottom action bar.
- ? Added page-level scroll containers for Modes, Base Tuning, Filtering, Combat Profile.
- ? Kept internal scroll where appropriate: rules list, stack columns, Helm/help workspace bodies, table horizontals.
- ? Replaced per-instance global wheel bindings with a single routed wheel system.
- ? Added cached scrollregion and width syncing.
- ? Preserved tab reuse.
- ? Added hidden-tab skip accounting.
- ? Moved major surfaces off canvas-rounded renderer.
- ? Simplified border/surface system.

Measured performance from repair report:

- ? Reused tab switches: about 1.49 ms average, 4.53 ms max after first-load reuse.
- ? Steady-state graph drawing: about 0.68 ms average.
- ? Biggest first-load hotspot: shared tuning bundle build, about 548.7 ms the first time a tuning tab was entered.

Scroll geometry verification:

- ? Modes: content height 683 vs viewport 386.
- ? Base: 897 vs 390.
- ? Filtering: 897 vs 390.
- ? Combat: 912 vs 390.
- ? Conditional Rules: 1057 vs 219.
- ? Effective Response Stack left: 1990 vs 137.
- ? Effective Response Stack right: 1084 vs 151.
- ? Help / Docs: 1055 vs 619.
- ? Helm: 757 vs 370.

Output: repaired build pointed to dist_repair1.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 2.5
V2 frontend rebuild

2.5 V2 frontend rebuild

Decision: The current UI stack had reached diminishing returns. The user wanted a complete UI redo, not another skin pass.

Chosen stack:

- ? PySide6 / Qt Widgets for the frontend.
- ? pyqtgraph for live/interactive graphs.
- ? Preserve existing backend/config/tuning/rule/bridge logic where possible.

Reasoning:

- ? Qt Widgets gives mature desktop layouts, splitters, scroll areas, tables, native UI patterns, cleaner rendering, and better professional polish.
- ? pyqtgraph is suitable for fast engineering/scientific plotting.
- ? Rebuilding the frontend allowed better looks and smoother performance while keeping the backend brain.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 2.6
Safe parallel V2 strategy

2.6 Safe parallel V2 strategy

V2 was intentionally designed as a parallel app, not a destructive rewrite.

Rules established:

- ? Current/legacy app remains intact and launchable.
- ? V2 has its own entry point, launcher, packaging target, and dist folder.
- ? Existing launchers should keep opening stable legacy until explicitly switched.
- ? V2 initially uses safe workspace/draft behavior.
- ? V2 should not silently overwrite original configs.
- ? V2 should read/import legacy but save separately until compatibility is proven.

Suggested structure:

```
/shared_core
  backend logic
  config parsing
  tuning math
  rule engine
  runtime bridge adapters
  common models / DTOs

/legacy_app
  current existing frontend
  untouched except safe shared-core import adjustments if required

/v2_app
  new PySide6 frontend
  new page classes
  new widgets
  new theme system
  new graph components
  new launcher / entry point
```

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 2.16
Help / Docs page

2.16 Help / Docs page

Purpose: built-in app guide/documentation.

Visible title: Help / Docs

Visible copy:

- ? "Search the built-in guide, browse by category, and keep the details you use most close at hand."

Controls:

- ? Search field placeholder: Search features, pages, or tuning terms.
- ? Sort dropdown: By Category.

Layout:

- ? Topics tree on left.
- ? Guide pane on right.

Topics tree categories visible:

- ? Advanced Pages
- ? Conditional Rules
- ? Effective Response Stack

- ? Helm
- ? Analysis
 - ? Graphs and Previews
 - ? Runtime Indicators
- ? Core Pages
 - ? Base Tuning
 - ? Combat Profile
 - ? Filtering
 - ? Modes
 - ? Profiles
- ? Diagnostics
 - ? Performance / Diagnostics
- ? Getting Started
 - ? Quick Start
- ? Reference
 - ? Tuning Glossary
- ? Workflow
 - ? Saving and Importing

Guide content example: Conditional Rules guide explains rules answer three questions:

1. What changes? Choose target axis, parameter, operation, value.
2. When does it run? Add mode gates or button gating if needed.
3. What signal condition does it watch? Pick reference axis, processing stage, measure type, comparator, and threshold.

Source-Preserved Block:

[hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 2.17 Performance / Diagnostics page](#)

2.17 Performance / Diagnostics page

Purpose: show timing and instrumentation so performance claims are verifiable.

Legacy/V2 diagnostics fields included:

- ? active page
- ? runtime state
- ? selected axis
- ? source file
- ? hidden page skips
- ? runtime path checks
- ? page build/switch times
- ? heartbeat timing
- ? graph draw timings
- ? startup total/config/widget timing

Example metrics visible in V2 screenshot:

- ? v2.active_page: perf
- ? v2.runtime: live
- ? v2.selected_axis: Yaw
- ? v2.source: V2 draft: hotas_bridge_config_v2.json
- ? v2.hidden_page_skips: 41689
- ? page build timings for base, combat, stack, filtering, modes, rules, help, perf.
- ? page switch timings.
- ? heartbeat average/max.
- ? graph draw average/max.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 4.1
Legacy architecture details

4.1 Legacy architecture details

Known/mentioned files:

- ? hotas_control_panel_app.py
 - ? Main legacy UI/app logic.
 - ? Received layout repair, page scrolling, centralized wheel routing, tab/perf instrumentation, compact footer, scroll tracking.
- ? app_performance.py
 - ? Performance coordination/refactor work.
- ? hotas_ui_theme.py
 - ? Surface rendering strategy, visual tokens, chrome/border behavior.
- ? Open-HOTAS-Control-Panel.cmd
 - ? Launcher.
- ? Edit-HOTAS-Curves.cmd
 - ? Launcher/editor helper.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 4.2
Legacy performance architecture

4.2 Legacy performance architecture

- ? Coordinated heartbeat instead of competing refresh timers.
- ? Adaptive runtime polling.
- ? Hidden/non-live areas polled less.
- ? Targeted dirty tracking per tab instead of whole-app snapshots.
- ? Preview math caching.
- ? Hidden-tab skip counters.
- ? Graph render timing.
- ? Scroll region instrumentation.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 4.3
V2 architecture target

4.3 V2 architecture target

Frontend:

- ? Python + PySide6 / Qt Widgets.
- ? pyqtgraph for live plots and response previews.

Architecture concepts:

- ? QMainWindow shell.
- ? Left sidebar navigation.
- ? Header/status/assistant area.
- ? Central stacked page container.
- ? Footer action/status bar.
- ? QScrollArea for tall pages.

- ? QSplitter/resizable layouts where useful.
- ? Lazy page construction.
- ? Reused page instances.
- ? Hidden pages skip expensive work.
- ? Scoped graph updates.
- ? Signals/slots for cross-component updates.
- ? Theme/QSS design system.

Recommended folder layout:

```
v2_app/
  main.py / app entry point
  ui/
  pages/
  widgets/
  adapters/
  services/
  models/
  theme/
shared_core/
  config models/parsing
  tuning math
  rule engine
  runtime bridge adapters
  common domain models
legacy_app/
  preserved original frontend
packaging/
  build scripts
  installer scripts
  icon assets
```

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 4.4
State management

4.4 State management

V2 uses a safe workspace/draft model:

- ? Source file visible: hotas_bridge_config_v2.json.
- ? Status chip: Workspace Copy.
- ? Saved state: Saved / Clean earlier.
- ? Runtime state: Idle / Live.
- ? Footer says Profile: Current Workspace.
- ? Changes update current workspace immediately and write out only when saved.
- ? Import Profile, Revert, Save Workspace available globally.
- ? Helm changes apply only to current workspace/draft until saved.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 4.5
Data flow inferred

4.5 Data flow inferred

1. Physical HOTAS input is read by runtime bridge.
2. Mapping page routes raw axes/buttons/hats to logical outputs and runtime vJoy outputs.
3. Base Tuning shapes underlying axis response.
4. Filtering applies smoothing and slew limits.
5. Modes determine Precision/Combat/Trigger/Zoom/Extra states.
6. Combat Profile applies combat/zoom-specific response modifiers.

7. Conditional Rules inject changes inline at stages.
8. Effective Response Stack shows full pipeline.
9. Live Monitor samples raw/final runtime values.
10. Profiles store selectable config sets.
11. Helm reads current tuning context and proposes workspace diffs.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 9.1
Key implementation briefs from this chat

9.1 Key implementation briefs from this chat

Important prompts were created for Codex:

1. Legacy Phase 3 layout/performance/render repair.
2. PySide6/Qt frontend rebuild using pyqtgraph.
3. Safe parallel V2 migration.
4. Ground-up V2 frontend overhaul with rounded corners and premium polish.
5. V2 correctness/semantics/premium polish update.
6. Final premium polish/productization pass.
7. Windows packaging/installer/icon setup.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 9.2
Important file/component names preserved

9.2 Important file/component names preserved

Legacy:

- ? hotas_control_panel_app.py
- ? app_performance.py
- ? hotas_ui_theme.py
- ? Open-HOTAS-Control-Panel.cmd
- ? Edit-HOTAS-Curves.cmd
- ? HOTASControlPanel.exe

V2/packaging expected:

- ? v2_app/
- ? shared_core/
- ? legacy_app/
- ? packaging/
- ? packaging/inno/
- ? packaging/build_release.ps1 or similar.
- ? dist_v2_release/
- ? installer_output/
- ? assets/app_icon.ico

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 9.3
Dependencies mentioned

9.3 Dependencies mentioned

- ? PySide6 / Qt Widgets.
- ? pyqtgraph.
- ? Possible packaging tools:
 - ? pyside6-deploy
 - ? PyInstaller one-folder build
 - ? Inno Setup installer

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 10.1
Legacy Phase 2: pixelated rounded corners

10.1 Legacy Phase 2: pixelated rounded corners

- ? Symptom: rounded corners looked jagged/pixelated.
- ? Likely cause: custom canvas-backed rounded renderer, aliased custom drawing, scaled assets, too many outline/shadow layers, DPI scaling mismatch.
- ? Fix: move major surfaces off canvas-rounded renderer, use cleaner frame-backed shells, reduce border/shadow spread, later switch to Qt/QSS border-radius in V2.
- ? Status: legacy repaired; V2 solved more cleanly.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 10.2
Legacy Phase 2: missing/broken scrollbars

10.2 Legacy Phase 2: missing/broken scrollbars

- ? Symptom: user could not view full page on tabs.
- ? Cause: bad scroll container structure and `bind_all()` wheel routing per instance.
- ? Fix: page-level scroll containers, centralized wheel routing, cached scrollregion/width sync.
- ? Status: repaired.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 10.3
Legacy Phase 2: slow tab switching

10.3 Legacy Phase 2: slow tab switching

- ? Symptom: tab switching became slower.
- ? Likely cause: rebuilding tab contents, canvas redraws, graph redraws, layout thrash.
- ? Fix: tab reuse instrumentation, reduced custom rendering, hidden-tab skip accounting.
- ? Status: improved; V2 later addressed fundamentally.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 10.4
Legacy Phase 2: bottom action bar clipping content

10.4 Legacy Phase 2: bottom action bar clipping content

- ? Symptom: bottom workspace actions covered/reduced usable content.

? Fix: footer made compact and explicitly reserved in layout.

? Status: repaired.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 10.5

V2 early: active sidebar page indicator wrong

10.5 V2 early: active sidebar page indicator wrong

? Symptom: Modes stayed highlighted even when on other pages.

? Fix: active page selection state must update correctly; final screenshots show correct active states.

? Status: fixed.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 10.8

V2 early: top-right statuses looked like buttons

10.8 V2 early: top-right statuses looked like buttons

? Symptom: V2 Draft/Clean/Runtime Live looked like Import/Save buttons.

? Fix: split status chips and action buttons visually.

? Status: final top-right cluster uses STATUS chips and ASSISTANT row.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 10.12

Icon generation workflow failure

10.12 Icon generation workflow failure

? Symptom: user asked for a second simplified icon variant and eventually got frustrated because output handling did not match request.

? Correct expected output: generate simpler bold icon and convert/export actual .ico file, not merely PNG/image preview.

? Remaining task: ensure simplified variant is converted to multi-size .ico and linked/placed at assets/app_icon.ico.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 12.1

Must-have features

12.1 Must-have features

? [] Safe parallel V2 frontend in PySide6.

? [] pyqtgraph graphs.

? [] Left sidebar navigation.

? [] Header with status cluster and Helm launcher.

? [] Footer with status and save/import/revert actions.

? [] Mapping page.

? [] Modes page.

? [] Base Tuning page.

? [] Filtering page.

? [] Combat Profile page.

? [] Profiles page.

- ? [] Conditional Rules page.
- ? [] Effective Response Stack page.
- ? [] Live Monitor page.
- ? [] Help / Docs page.
- ? [] Perf / Diagnostics page.
- ? [] Helm overlay assistant.
- ? [] Runtime status indicators.
- ? [] Save/import/revert workspace model.
- ? [] App packaging/installer.
- ? [] App icon.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 12.2
Nice-to-have / later features

12.2 Nice-to-have / later features

- ? [] Flight Recorder / hindsight buffer.
- ? [] Overlay toggle with keybind.
- ? [] Richer Helm analysis text.
- ? [] More detailed profile editing.
- ? [] Full native rule editor.
- ? [] Export/import profile library.
- ? [] More animation polish.
- ? [] About/update/check-for-updates panel.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 12.3
Implementation order

12.3 Implementation order

1. Preserve legacy app.
2. Extract/define shared core interfaces.
3. Build V2 Qt shell.
4. Build design system/QSS.
5. Implement mapping/workspace/profile state.
6. Implement tuning pages and graphs.
7. Implement rules/stack/live monitor.
8. Implement Helm overlay.
9. Implement Help/Docs/Perf.
10. Add polish/copy/animations.
11. Package with installer and icon.
12. Test end-to-end.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ### 12.4
Testing/debug checklist

12.4 Testing/debug checklist

- ? [] Legacy app still launches.

- ? [] V2 launches separately.
- ? [] Active nav state correct for every page.
- ? [] All pages scroll if needed.
- ? [] No overlap/clipping at common window sizes.
- ? [] Base linear reference is truly linear.
- ? [] Graphs update without whole-page jank.
- ? [] Effective Response Stack does not twitch.
- ? [] Hidden pages skip work.
- ? [] Runtime live/idle indicators accurate.
- ? [] Save workspace writes correct file.
- ? [] Revert restores last saved/imported state.
- ? [] Import Profile works safely.
- ? [] Helm apply/revert only affects workspace/draft until saved.
- ? [] Installer installs/uninstalls cleanly.
- ? [] App icon appears on EXE, installer, Start Menu, taskbar.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ## 13.
Unknowns / Gaps

13. Unknowns / Gaps

- ? Exact original source code is missing.
- ? Exact internal data schema for hotas_bridge_config_v2.json not shown.
- ? Exact runtime bridge implementation not shown.
- ? Exact vJoy library/API calls not shown.
- ? Exact controller polling method not shown.
- ? Flight Recorder/overlay details mentioned but not fully shown in screenshots.
- ? No complete packaging scripts shown yet.
- ? Simplified app icon PNG exists conceptually, but final .ico conversion path must be completed.
- ? Exact animation implementation not shown.
- ? Exact Helm diagnostic algorithm not provided; only UI behavior and example output.
- ? Exact profile JSON schema unknown.
- ? Exact rule engine schema unknown beyond visible fields.

Questions to answer later:

1. What is the full config schema?
2. What Python modules handle vJoy and physical HOTAS input?
3. What runtime bridge process/API exists?
4. What exact package/version requirements are needed?
5. Should installer be per-user or per-machine?
6. Should V2 eventually become default launcher?
7. Should legacy be kept installed side-by-side forever?

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ## 14.
Codex / AI Rebuild Prompt

14. Codex / AI Rebuild Prompt

You are rebuilding HOTAS Control Panel V2 from recovery notes.

The original app was erased. Use these notes as the blueprint. The goal is to rebuild a polished PySide6 / Qt Widgets desktop app for HOTAS/vJoy mapping, tuning, live monitoring, profiles, conditional rules, effective response stack inspection, and Helm assistant diagnostics.

Do not rebuild the old legacy custom-painted UI. Build the V2 architecture.

Core stack:

- Python
- PySide6 / Qt Widgets
- pyqtgraph for all live/interactive graphs
- Shared backend/core modules for config, tuning math, rule engine, runtime bridge adapters, profiles, and workspace state

Preserve these major pages:

- Mapping
- Modes
- Base Tuning
- Filtering
- Combat Profile
- Profiles
- Conditional Rules
- Effective Response Stack
- Live Monitor
- Help / Docs
- Perf / Diagnostics
- Helm overlay assistant

Global UI requirements:

- Dark premium engineering-software theme
- Rounded corners throughout using Qt/QSS/native styling
- Left sidebar with active page highlight and runtime status at bottom
- Header with title, polished subtitle, STATUS chips, and ASSISTANT/Helm launcher
- Footer with status, page/axis/profile/source details, Import Profile, Revert, Save Workspace
- No ugly dark text strips everywhere
- No debug/prototype wording
- Product-grade copy
- Smooth page switching and no visible redraw jank
- Hidden pages must skip expensive work
- Graph updates must be scoped locally
- Performance diagnostics must remain available

Implement safe workspace model:

- Use a workspace copy/draft file, e.g. hotas_bridge_config_v2.json
- Changes update the workspace immediately but only write permanently when user saves
- Import Profile, Revert, Save Workspace actions must be safe
- Helm changes apply only to current workspace until saved

Implement Mapping page:

- Routing Overview: Axis Routes 6, Button Routes 15, Hat Routes 1
- Live Route Summary with mappings such as Roll Raw axis 1 -> X -> X(axis1), Pitch Raw axis 2 -> Y -> Y(axis2), Throttle Raw axis 3 -> Z -> Z(axis3), Yaw Raw axis 6 -> RZ -> RX(axis4), Aux1 Raw axis 7 -> SL0 -> RY(axis5), Aux2 Raw axis 8 -> RX -> RZ(axis6)
- Axis Routing table with Function, Raw Axis, Logical Output, Runtime vJoy, Invert, Live Raw, Live Output
- Button Routing table with Add Route, Remove Selected, Reset 1:1
- Hat Routing table with Add Hat, Remove Selected

Implement Modes page:

- Precision Mode with Buttons field, default 0
- Combat Mode with Trigger Buttons, Zoom Buttons default 5, Extra Buttons, Stack Mode multiply
- Live Mode State showing Precision/Combat/Trigger/Zoom/Extra/Stack states
- Mode Notes with polished product copy

Implement Base Tuning page:

- Mapped axes list Roll, Pitch, Throttle, Yaw, Aux1, Aux2
- Parameters: Curve Mode, Curve Strength, Deadzone, Anti-Deadzone, Hysteresis, Output Scale, Max Output
- pyqtgraph response preview
- Linear reference line must be true $y=x$ and never transformed by the curve math
- Live Snapshot and Guidance cards

Implement Filtering page:

- Parameters: Center Alpha, Edge Alpha, Same Slew Limit, Reverse Slew Limit
- Step preview graph showing input vs filtered output
- Live Snapshot and Guidance

Implement Combat Profile page:

- Parameters: Combat Curve, Combat Scale, Combat Center Alpha, Combat Edge Alpha, Combat Same Slew, Combat Reverse Slew
- Combat response graph
- Guidance card with warning/caution/extreme states where needed

Implement Profiles page:

- Profile Library with Built-In Presets: Balanced Flight, Precision Tracking, Aggressive Combat, Smooth Cinematic
- Personal Profiles with Current Workspace active
- Profile Detail, Setup Summary, Profile Feel, Profile Actions
- Buttons: Save Current As New, Import JSON, Duplicate, Export JSON, Use This Profile, Rename, Delete

Implement Conditional Rules page:

- Status chips: rules, active, blocked, disabled
- Action row: Add Rule, Edit Selected, Duplicate, Enable, Delete
- Rule List and Rule Detail split
- Example rule: Yaw 0.75 | Roll > 0.35, disabled, target Yaw, parameter Output Scale, operation Set, value 0.75, injects at Base Output Limits, mode gate Always
- Rule Logic explanatory card
- Keep production rule logic safe; full advanced editor can come later if needed

Implement Effective Response Stack page:

- Axis selector
- Live status
- Signal Chain with stage cards Raw Input, Center Conditioning, Curve / Shape, etc.
- Stage cards show IN/OUT/DELTA, live/active badges, input/output bars, description
- Raw vs Final graph with live marker
- Mode State, Current Stack Summary, Selected Stage, Rule Driver Values cards
- Fix/avoid periodic twitch: reuse widgets and update values in place; do not rebuild cards on heartbeat

Implement Live Monitor page:

- Monitor Controls with axis selector and Show raw and output together checkbox
- Raw Input Trace graph
- Raw vs Final Overlay graph
- Live State and Buttons/Hats cards
- Axis Levels visualization with raw blue and final green bars for Roll/Pitch/Throttle/Yaw/Aux1/Aux2
- HOTAS Buttons B1-B15
- Mapped Output Buttons Out1-Out20

Implement Help / Docs:

- Search field
- Sort dropdown By Category
- Topics tree with Advanced Pages, Analysis, Core Pages, Diagnostics, Getting Started, Reference, Workflow
- Guide pane with product documentation articles

Implement Helm overlay:

- Not a sidebar page
- Top-right assistant launcher opens overlay/modal
- Background app dimmed/blurred/de-emphasized
- Green pulsing indicator: Helm is active / Context-linked assistant
- Sections: What's wrong?, What I'd change, What I found, Apply/Revert
- Symptom chips: Can't hold aim steady, Too twitchy near center, Overshoots target, Combat mode feels sluggish, Reversals feel sticky, Hard to track smoothly
- Actions: Analyze, Review Changes, Cancel, Apply Selected Changes, Revert Last Helm Changes
- Text tone must be polished and product-grade

Implement Perf / Diagnostics:

- Show active page, runtime state, selected axis, source file, hidden page skips, runtime path checks, page build/switch timings, heartbeat timing, graph draw timings, startup timings

Packaging:

- Build release executable/folder
- Create proper Windows installer using Inno Setup or equivalent
- Create Start Menu shortcut, optional Desktop shortcut, uninstall entry
- Install app files to Program Files or LocalAppData Programs
- Store user data/config/profiles/logs in AppData/LocalAppData, not beside executable
- Use app icon assets/app_icon.ico
- Create both detailed icon for large sizes and simplified bold icon for taskbar/start-menu

Testing requirements:

- No overlap or clipping
- Every page scrolls correctly
- Graphs smooth
- Hidden pages skip work
- Effective Response Stack stable
- Active sidebar state correct
- Linear reference line truly linear
- Runtime live/idle indicators accurate
- Save/import/revert safe
- Helm overlay safe and performant
- Installer app icon appears correctly

Build this as polished, premium, shippable desktop software. The target reaction is: "DAMN THIS IS NICE SOFTWARE."

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / ## 15.
Immediate Recovery Notes From This Chat's Last Turn

15. Immediate Recovery Notes From This Chat's Last Turn

The user became frustrated because a requested instruction was not followed. The important correction is:

- ? If asked to create a Canvas recovery document, create the Canvas document immediately and do not continue prior icon-generation work.
- ? If asked for an .ico, produce an actual .ico file, not only a generated image.
- ? For the simplified icon, create a multi-size ICO from the simple PNG and save it under a clear filename such as HOTAS_Control_Panel_V2_Taskbar_Icon.ico.

This document was created to satisfy the recovery-document instruction and preserve the HOTAS V2 rebuild details from the chat.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md /
2.9 Performance / Latency Refactor

2.9 Performance / Latency Refactor

Why it was needed

The app became sluggish after rich live visualization features were added. User explicitly wanted:

- ? CPU and GPU use minimized
- ? system latency cut down to an absolute minimum
- ? whole app to feel smooth and immediate

Major Phase 1 performance changes recorded in chat

- ? coordinated heartbeat instead of competing timers
- ? adaptive runtime polling
- ? hidden/non-live areas no longer polled at the same cost as live views
- ? dirty-state tracking reduced from whole-app snapshot churn to targeted refresh work
- ? preview math cached for references/curves/response previews/feel/guidance output
- ? graphs accept signature hints so marker/live updates do not pay full geometry-signature cost
- ? Effective Response Stack freeze actually freezes the view
- ? stack cards and rule cards keep persistent structure and skip reconfiguring unchanged widgets
- ? perf panel improved with diagnostics for visible tab, runtime mode, pending work, scheduler timing, cache behavior

Measured notes reported

- ? steady stack refresh ~1.7 ms after layout in smoke run
- ? stack.build_state ~0.10 ms
- ? rules.render_cards ~0.22 ms
- ? graph.draw_plot ~0.45 ms average in mixed smoke
- ? remaining hotspots: per-tab preview recomputation and dirty-state capture during edits ~9–11 ms in source-mode smoke tests

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md / ##
4. Software Architecture / Technical Design

4. Software Architecture / Technical Design

Known files/modules mentioned explicitly

- ? hotas_control_panel_app.py
- ? app_performance.py
- ? HOTASControlPanel.exe
- ? Open-HOTAS-Control-Panel.cmd
- ? Edit-HOTAS-Curves.cmd

Architecture direction inferred

Python desktop app with a central application file plus supporting modules. The later refactor strongly suggests the app uses:

- ? a main UI shell
- ? per-tab views/components
- ? shared state / runtime control logic
- ? performance helper / scheduler module
- ? persistent graph preview components
- ? structured subsystems for advanced features like Helm

Important architectural principles explicitly discussed

- ? do not cram advanced logic into one main app file
- ? Helm should be modular:
 - ? engine/service
 - ? symptom library
 - ? fix/recommendation library
 - ? types/models
 - ? context-analysis helpers
- ? app should use a retained/component-like architecture rather than visibly redrawing pages in chunks
- ? UI should use persistent containers/surfaces where possible
- ? heavy live work should be decoupled from core control path

Data flow / state ideas

Likely flow for axis tuning:

1. raw controller input
2. center conditioning
3. curve shaping
4. output limits
5. filtering
6. mode modifiers
7. rule injections
8. final output / vJoy / downstream control output

This was made explicit in Effective Response Stack discussions.

Saving/loading configuration

Not fully detailed, but clearly present. Important note from Helm:

- ? Helm applies changes to in-memory current state only
- ? user must still save manually

State management desires

- ? persistent structures for stack cards and rule cards
- ? avoid full-page rebuilds

- ? central update heartbeat
- ? hidden tabs/workspaces should suspend or throttle heavy live work
- ? graph geometry caching
- ? only selected axis stack recomputed/rendered

Planned architecture improvements

- ? design tokens / visual system
- ? motion tokens / motion system later
- ? centralized scheduler/heartbeat
- ? performance debug layer
- ? improved modularization for Helm

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md /
 ### Concrete implementation details described from Codex update

Concrete implementation details described from Codex update

- ? coordinated heartbeat for UI/dirty/runtime updates
- ? adaptive runtime polling
- ? targeted per-tab dirty-state tracking instead of whole-app snapshot churn
- ? caching for:
 - ? linear references
 - ? static curves
 - ? dynamic response previews
 - ? feel/guidance output
- ? graph signature hints so marker/live updates avoid full geometry cost
- ? Effective Response Stack freeze now freezes the view
- ? persistent stack card and rule card structures
- ? perf diagnostics expanded

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md /
 ### Bug/problem: app became sluggish / delayed

Bug/problem: app became sluggish / delayed

Symptom

Controls felt sluggish/delayed after adding richer visualizations.

Likely causes discussed

- ? Effective Response Stack doing too much live work
- ? graphs rebuilding too much
- ? animations too expensive
- ? summaries/rule status recalculating too often
- ? hidden tabs possibly still updating
- ? control path too coupled to UI work

Fixes

- ? coordinated heartbeat
- ? adaptive polling

- ? hidden area work reduction
- ? graph/path caching
- ? persistent structures
- ? freeze mode improvements
- ? diagnostics

Result

Significant Phase 1 performance improvement.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md /
 ### Must-have features

Must-have features

- ? Main shell with dark technical UI
- ? Tabs:
 - ? Modes
 - ? Base Tuning
 - ? Filtering
 - ? Combat Profile
 - ? Conditional Rules
 - ? Effective Response Stack
- ? per-axis tuning tables
- ? graph preview panels on right
- ? grouped columns/sections with tooltips
- ? mode system (precision/combat stack)
- ? Conditional Rules editor with grouped sections and plain-English preview
- ? Effective Response Stack with one-axis live chain and inline rule injection
- ? Helm workspace launched from top-right
- ? save/load current state
- ? performance-optimized update architecture

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md /
 ### Nice-to-have / later features

Nice-to-have / later features

- ? preset architecture
- ? Quick Tune / onboarding setup
- ? stronger Helm intelligence / fingerprint matching
- ? rule simulator
- ? export/screenshot/reporting
- ? richer docs drawer
- ? motion system overhaul

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md /
 ### UI pieces to recreate

UI pieces to recreate

- ? shell/header
- ? top tab strip
- ? right-side graph panels
- ? action bar
- ? rule editor card system
- ? stack cards and arrows
- ? Helm workspace overlay mode
- ? perf/help/status controls top-right

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md /
Likely files/modules needed

Likely files/modules needed

- ? main app shell/app file
- ? performance/scheduler module
- ? graph/preview module(s)
- ? rule engine/editor module
- ? response stack inspector module
- ? Helm engine/module set
- ? design token / styling layer
- ? persistence/config module

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md /
Suggested rebuild order

Suggested rebuild order

1. Core app shell and performance-friendly state/update architecture
2. Base Tuning / Filtering / Combat / Modes pages with graph panels
3. Conditional Rules editor and rule logic
4. Effective Response Stack
5. Helm workspace v1
6. performance polish
7. visual modernization / motion modernization
8. preset architecture / Quick Tune / docs / smarter Helm

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md /
Testing/debugging checklist

Testing/debugging checklist

- ? verify axis mapping and updates
- ? verify graph updates without lag
- ? verify hidden tabs do not keep full live work
- ? verify stack only renders selected axis
- ? verify freeze truly freezes stack state
- ? verify rule states (active/blocked/etc.) display correctly

- ? verify Helm apply/revert only touches in-memory state
- ? verify no overlap/clipping/layout crowding
- ? verify performance metrics stay low

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md / ##

13. Unknowns / Gaps

13. Unknowns / Gaps

Missing/unclear details

- ? exact dependency list / packages
- ? exact project folder structure besides files mentioned
- ? exact persistence file format
- ? exact graph rendering library / toolkit
- ? exact UI toolkit/framework details
- ? exact launcher packaging workflow
- ? exact styles/colors/tokens after later visual phases
- ? exact code for curves and transformations as implemented in final app

Screenshots needing manual review later

All screenshots should be manually reattached and used to refine:

- ? exact spacing
- ? exact labels
- ? exact colors
- ? exact panel widths
- ? exact chips/button text
- ? exact graph styling

Feature references not fully explained

- ? full overlay/flight recorder UI
- ? preset architecture details
- ? Quick Tune specifics
- ? help/docs drawer content layout
- ? export/reporting format
- ? eventual rule simulator behavior

Questions to answer later

- ? what UI framework was used exactly?
- ? what config file format was used?
- ? how were controller devices enumerated and mapped?
- ? what graph library/path rendering system was used?
- ? what exact design tokens/colors should be used in final rebuild?

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md / ##

14. Codex / AI Rebuild Prompt

14. Codex / AI Rebuild Prompt

Use this as a rebuild-oriented prompt for Codex or another coding agent:

Rebuild the HOTAS Control Panel desktop app described in these recovery notes.

This is a premium enthusiast-grade HOTAS tuning and diagnostics tool, not a generic settings page. The app must support deep per-axis response tuning, mode-specific behaviors, conditional rules, live graph previews, a full one-axis Effective Response Stack inspector, and a diagnostic workspace called Helm.

Core architecture requirements

- ? Build a persistent desktop app shell with a dark technical control-console aesthetic.
- ? Optimize for low CPU/GPU use and minimal control latency.
- ? Use retained/persistent UI surfaces instead of visibly redrawing page chunks.
- ? Hidden tabs/workspaces must suspend or heavily throttle expensive live work.
- ? Graphs must cache geometry where possible and update live markers independently.
- ? The control-processing path must stay responsive even when rich diagnostics are open.

Main tabs/features

1. Modes

- ? Precision Hold Buttons
- ? Combat Trigger Buttons
- ? Combat Zoom/Aim Buttons
- ? Combat Extra Buttons
- ? Precision + Combat Stack Mode
- ? Current Behavior Summary
- ? tooltip-style help

2. Base Tuning

- ? per-axis fields for Curve, k, Deadzone, Anti-DZ, Hysteresis, Scale, Max, Precision, Invert
- ? grouped column families
- ? graph preview on right
- ? feel summary / tuning guidance

3. Filtering

- ? per-axis Center Alpha, Edge Alpha, Same Slew, Reverse Slew
- ? grouped columns
- ? dynamic response preview graph
- ? feel summary / caution guidance

4. Combat Profile

- ? per-axis combat-specific curve/filtering values
- ? graph preview and overlays

5. Conditional Rules

- ? grouped editor with Action / When / Condition sections
- ? live plain-English rule preview sentence
- ? strong enabled/disabled state
- ? validation/conflict indicators
- ? runtime status chips
- ? tooltip help for advanced fields

6. Effective Response Stack

- ? one selected axis at a time
- ? vertical live stage cards with arrows/connectors
- ? subtle green animated flow down arrows

- ? stages: Raw Input → Center Conditioning → Curve → Base Output Limits → Filtering → Mode Modifiers → inline Rule Injections → Final Output
- ? graph on right showing Raw vs Final with live marker
- ? selected stage detail panel
- ? Current Stack Summary
- ? Rule Driver Values panel
- ? Freeze/Resume that freezes actual state
- ? strong distinction between main stages and injected rule cards

7. Helm

- ? launched from top-right, not in main tab strip
- ? opens a right-side slide-out workspace covering about 70% width
- ? background dims and blurs while open
- ? "Helm is active" bar fades in with subtle green pulse
- ? first-person voice
- ? natural language symptom input
- ? prompt chips
- ? 0–2 follow-up questions when needed
- ? structured diagnosis engine underneath
- ? confidence indicator
- ? context notes using stack/rules/modes/feel summary
- ? exact before/after diffs with checkboxes
- ? apply selected changes to in-memory state only
- ? revert last Helm changes
- ? no auto-save
- ? do not let Helm edit/create rules in v1

Visual design requirements

- ? dark premium control-console aesthetic
- ? visible but disciplined rounded corners
- ? soft shadows and strong surface hierarchy
- ? precise spacing, zero overlap, clean alignment
- ? modernized buttons, inputs, tabs, cards, graph panels
- ? not bubbly, not generic SaaS, not toy-like

Motion / rendering requirements

- ? use transform/opacity-style transitions later where possible
- ? persistent surfaces, not page-like redraws
- ? no visible chunk-by-chunk panel loading
- ? smooth, retained-feeling UI behavior

Performance requirements

- ? minimum practical CPU/GPU use
- ? minimal end-to-end control latency
- ? central update heartbeat / scheduler
- ? hidden/inactive content suspension
- ? graph/path caching
- ? diagnostics/perf mode for future regression detection

Build order suggestion

1. shell + performance foundation
2. base/filter/combat/modes pages with graph previews
3. conditional rules

4. effective response stack
5. Helm workspace
6. visual modernization pass
7. motion modernization pass
8. presets / Quick Tune / smarter Helm / docs drawer / simulator later

Tone/UX requirements

- ? serious, instrument-like, enthusiast-grade
- ? highly inspectable and explainable
- ? trust-building visuals
- ? rich diagnostics but still understandable

Rebuild from these notes as faithfully as possible, preserving feature names, labels, relationships, and the app's overall identity.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md
/ ### Confirmed architecture decisions from the chat

Confirmed architecture decisions from the chat

The live overlay should be implemented without destroying the existing Flight Recorder page. The assistant proposed, and the later screenshots strongly imply, the following architecture path:

Shared overlay engine

A shared overlay core should handle:

- ? telemetry history buffering,
- ? axis inclusion,
- ? axis colors,
- ? time window / history duration,
- ? graph scaling / layout,
- ? legend rendering,
- ? live value rendering,
- ? line/trace generation.

Two rendering/output paths

1. Replay / export overlay rendering
 - ? Used when compositing overlay into captured recordings.
2. Live transparent overlay window rendering
 - ? Used for detached, real-time on-screen overlay.

Suggested shared modules/components discussed

- ? telemetry buffer / history ring buffer
- ? overlay style/config model
- ? overlay trace builder / graph layout logic
- ? overlay renderer core
- ? replay/export compositor adapter
- ? live overlay window adapter

Window behavior for live overlay

The live overlay should be:

- ? separate top-level window,
- ? frameless / borderless,

- ? transparent background,
- ? always-on-top,
- ? click-through,
- ? able to show/hide independently of main app.

Main UI entry point decision

- ? Launch/control entry point should live in Live Monitor.
- ? Flight Recorder should remain focused on capture/replay/export.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md
/ ### State / workflow behavior inferred from screenshots

State / workflow behavior inferred from screenshots

Recorder states

Visible examples:

- ? Ready
- ? Hotkey armed
- ? Buffering
- ? Clip hotkey armed
- ? Recording
- ? Saved
- ? Live
- ? Unsaved
- ? Workspace Copy

This implies the app had a structured state model around:

- ? workspace dirty/saved state,
- ? recorder armed/running/buffering state,
- ? overlay active state,
- ? and live telemetry/runtime status.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md
/ ### Configuration persistence inferred

Configuration persistence inferred

Visible strings suggest saved/working files and preset/workspace relationships:

- ? Preset profile: Balanced Flight
- ? Workspace copy: hotas_bridge_config_v2.json

Inference: The app likely stored JSON-based workspace/config files and supported working copies or modifiable clones of presets.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md
/ ### Framework / language

Framework / language

No framework or programming language is explicitly confirmed in this chat.

Inference: Because of the custom desktop UI, detached overlay window, cards, dialogs, and Windows-native appearance, the app may have been implemented in something like Qt / PySide / PyQt or another desktop UI framework. However, this is not confirmed and

should not be treated as recovered fact.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md
/ ### Input handling / controller data model

Input handling / controller data model

Direct implementation code is not shown, but the UI and descriptions imply the app had at least two distinct streams:

- ? raw HOTAS input
- ? final vJoy output

This is explicitly visible in Live Monitor description text.

Inference: The app likely read physical controller input, applied mapping/tuning/transforms, then exposed/emitted a processed output stream through vJoy or equivalent virtual joystick layer.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md
/ ### Profiles / presets / workspaces

Profiles / presets / workspaces

Visible concepts:

- ? Preset
- ? Saved / Unsaved
- ? Workspace Copy
- ? Save Workspace
- ? Revert
- ? Preset profile: Balanced Flight
- ? Custom live overlay preset

This strongly suggests:

- ? reusable profiles/presets,
- ? mutable workspace copies,
- ? saved vs unsaved state management,
- ? and per-feature settings persistence.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md
/ ### Files/modules likely needed

Files/modules likely needed

Recovered from architecture discussion:

- ? telemetry buffer / history ring buffer module
- ? overlay config/style model
- ? trace builder / layout module
- ? renderer core
- ? replay/export compositor
- ? live overlay window module
- ? Flight Recorder page/module
- ? Live Monitor page/module

- ? overlay configuration dialog/module
- ? workspace/profile persistence module
- ? recording library management module
- ? clip preview/player module

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md
/ ### Recommended implementation order

Recommended implementation order

1. Recreate the main desktop shell and sidebar/header/footer layout.
2. Rebuild Flight Recorder page structure and static fields.
3. Rebuild Live Monitor page structure and graph placeholders.
4. Implement shared telemetry/overlay core.
5. Recreate export/replay overlay path for Flight Recorder.
6. Implement detached live overlay window using shared core.
7. Add Configure dialog and persistence.
8. Add forward capture and buffered capture trigger modes.
9. Recreate recording library and clip preview playback flow.
10. Add Helm icon/title treatment.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md
/ ### Testing / debugging checklist

Testing / debugging checklist

- ? Verify overlay colors match between live overlay and exported clips.
- ? Verify live overlay toggle hotkey works reliably.
- ? Verify recorder hotkey works for both forward and buffered modes.
- ? Verify Flight Recorder page remains visually intact after feature additions.
- ? Verify clip preview loads saved files correctly.
- ? Verify library refresh/sort behavior.
- ? Verify workspace save/revert and unsaved indicators.
- ? Verify overlay placement/opacity config persists.
- ? Verify click-through + always-on-top behavior on Windows.
- ? Verify attached display logic for overlay placement.

Source-Preserved Block:

hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md
/ ## 13. Unknowns / Gaps

13. Unknowns / Gaps

Details not recoverable from this chat

- ? Exact programming language/framework
- ? Exact project file structure and filenames
- ? Exact controller hardware model(s)
- ? Exact vJoy integration code or libraries

- ? Exact capture/encoding pipeline implementation
- ? Exact graph rendering implementation
- ? Exact workspace/preset JSON schema
- ? Exact overlay compositing code
- ? Any math for response curves or filtering
- ? Any button/hat mapping logic details

Screenshots needing manual review if higher precision is required

All screenshots should be manually reattached and zoomed if exact field text, spacing, or tiny labels are needed. This document preserves what is legible at chat resolution, but finer UI details may require direct image inspection in the original files.

Features referenced but not fully explained

- ? Mapping page
- ? Profiles page
- ? Modes page
- ? Base Tuning
- ? Filtering
- ? Combat Profile
- ? Conditional Rules
- ? Effective Response Stack
- ? Help / Docs
- ? Perf / Diagnostics
- ? Helm assistant capabilities beyond branding/presence

Questions future-you should answer later

- ? What exact desktop framework was used?
- ? What capture library or renderer was used for Flight Recorder?
- ? How were raw input and final output represented internally?
- ? What exact preset/workspace file formats existed?
- ? Was the live overlay always display-attached, or could it target a specific window?
- ? Were raw/final/both overlay source modes implemented beyond Final output?
- ? How were clips indexed in Recording Library?

Second Sweep Addendum: Source Code and Dependency Gaps

Confirmed Facts

The second comparison found explicit source statements about what code was not present and which concrete artifacts were named. These are preserved because they prevent a future rebuild from assuming the recovery notes contained implementation code they did not contain.

Source-Preserved Block:

`hotas_control_panel_recovery_notes_chat_v_2_rebuild_and_productization.md / implementation-details preface`

9. Code Snippets / Implementation Details

No full production source code snippets were provided in this chat. The chat contains implementation briefs, architecture prompts, Codex result reports, file names, and UI screenshots.

Source-Preserved Block:

`hotas_control_panel_recovery_notes_chat_v_2_flight_recorder_live_overlay_and_helm.md`

/ direct snippets

9. Code Snippets / Implementation Details

Direct code snippets present in this chat

No literal source code was pasted in this chat thread.

Source-Preserved Block:

**hotas_control_panel_recovery_notes_chat_v_2_hotas_control_panel_helm_recovery.md /
implementation artifacts and dependencies**

9. Code Snippets / Implementation Details

No large raw code snippet blocks from the user were included directly in this chat for the HOTAS app itself.

Concrete implementation artifacts mentioned

- ? hotas_control_panel_app.py
- ? app_performance.py
- ? launchers:
 - ? Open-HOTAS-Control-Panel.cmd
 - ? Edit-HOTAS-Curves.cmd
- ? packaged executable:
 - ? HOTASControlPanel.exe

Dependencies/setup

No package list explicitly stated in this chat. This is a gap for later recovery.
